Day 131: Contrastive Learning Basics: a Real Positive/Negative Pair Demonstration

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 19, Day 6 | Complete study guide

Watch: Contrastive Learning with SimCLR | Deep Learning Animated (Deepia) | Contrastive Learning of visual representations - SimCLR (paper illustrated) (AI Bites)

Script: `day131_contrastive_learning.py` · every number in this guide is copied by program from `day131_run_output.txt`, the real output of that script.

Days 126–130 learned by rebuilding pixels. Contrastive learning rebuilds nothing: it makes two random views of each image and trains a network to tell which view belongs with which. Written from scratch in NumPy (SimCLR's NT-Xent loss, with its gradient derived by hand and checked to 3.0e-10), trained on 20,000 MNIST images with no labels. With only 10 labelled images per digit, a linear classifier on its 128 numbers scores 0.8157, against 0.6078 on raw pixels and 0.7100 on Day 126's autoencoder code. Two honest complications are measured too: the best temperature depends on how many labels follow, and which views are used matters a great deal.

0. Where today fits

DEFINITION — start here, no prior knowledge assumed

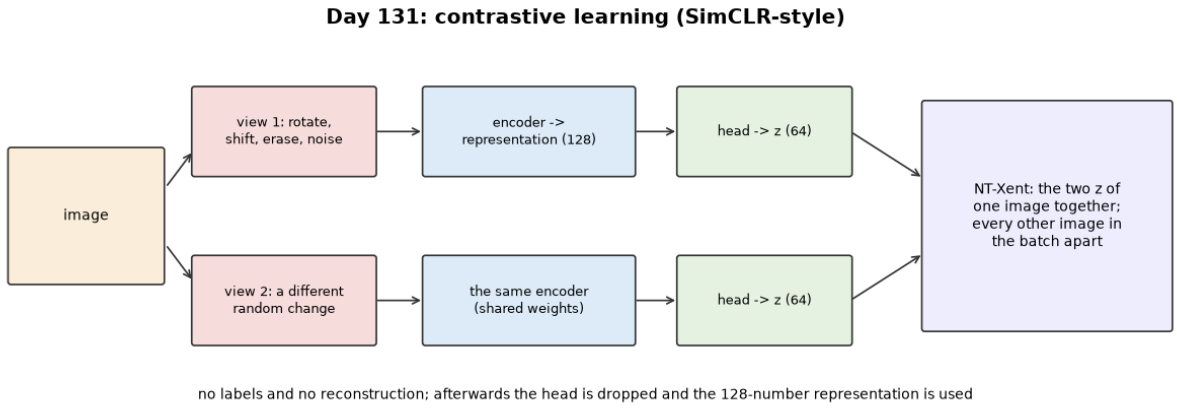
Self-supervised learning invents a task whose answers come free with the data. **Contrastive learning** is one such task: take an image, make two randomly changed copies (**views**), and train a network so that the two views of the same image land close together (a **positive pair**) while views of different images land far apart (**negative pairs**).

What survives every random change is what the image *is*, not its exact pixels. The network's inner layer, the **representation**, is then reused for a real task with few labels.

REAL-WORLD SCENARIO

A child shown photos of the family dog from different angles, in different light, half hidden behind a chair, learns what makes it *that* dog without anyone naming its breed. Later one word, "terrier", is enough to label all of them.

SYSTEM DIAGRAM



Two views of one image through the same network; the loss pulls their projections together and pushes the rest of the batch away.

Today's part for the week's build

`day131_exercise.py` is the views, the loss, training and the representation, checked by hand. Its real output:

ACTUAL OUTPUT

```
a view with every change switched off equals the image: True
no change keeps the pixel at row 3, column 13: True
two images, four views: loss 1.8680; row 0 picks its partner with probability 0.5341; by hand
    e^1.6 / (e^0 + e^1.6 + e^1.2) = 0.5341
gradient against finite differences: largest gap 3.9e-10
64 random images, a small network: loss 4.211 after epoch 1, 4.122 after 8; representation shape
    (64, 32)
```

Data: MNIST (cached by Day 126): the first 20,000 images train every network without labels; images 60,000–69,999 are the test set. Labels are used only to judge. Day 126's network parts and Adam, Day 123's PCA, Day 119's distances and Day 116's softmax regression are imported unchanged.

How to read each section

Step	What it gives you
DEFINITION	What the idea is, in plain words. Start here.
REAL-WORLD SCENARIO	An everyday situation where the same idea shows up.
MATHEMATICS	The formula, when the section introduces one.
SPECIFICATION	The exact setup of the experiment.
IMPLEMENTATION	The real code from the script.
ACTUAL OUTPUT	What the script really printed, copied unchanged.
What it means	How to read the numbers, and what they do and do not show.
Why choose this? / Why NOT this?	When the idea is the right tool, and what it costs.

1. By hand: the loss for two images

DEFINITION — start here, no prior knowledge assumed

NT-Xent (normalised temperature-scaled cross-entropy) asks each view to pick out its partner among every other view in the batch. It is a softmax classification whose 'classes' are the other views.

REAL-WORLD SCENARIO

A line-up: you are shown one photo and must point to the other photo of the same person among several strangers.

MATHEMATICS

$$u_i = z_i / |z_i| \qquad s_{ij} = u_i \cdot u_j / \tau$$

$$L_i = -\log \left[e^{s_{i,p(i)}} / \Sigma_{j \neq i} e^{s_{ij}} \right] \qquad \text{loss} = \text{mean over all } 2N \text{ views of } L_i$$

$p(i)$ = the other view of the same image; τ = the temperature

SPECIFICATION

Projections	4 unit-length 2-D vectors: rows 0 and 2 are one image's views, rows 1 and 3 another's
Temperature	0.5

IMPLEMENTATION

```
def nt_xent(Z, tau):
    n2 = len(Z); N = n2 // 2; norm = np.linalg.norm(Z, axis=1, keepdims=True) + 1e-12; U = Z /
        norm
    S = U @ U.T / tau; np.fill_diagonal(S, -np.inf)
    pos = np.r_[np.arange(N, n2), np.arange(N)]
    m = S.max(1, keepdims=True); E = np.exp(S - m); P = E / E.sum(1, keepdims=True)
    loss = float(np.mean(-np.log(P[np.arange(n2), pos] + 1e-30)))
    dS = P.copy(); dS[np.arange(n2), pos] -= 1; dS /= n2
    dU = (dS + dS.T) @ U / tau
    dZ = (dU - U * (U * dU).sum(1, keepdims=True)) / norm
    return loss, dZ, P[np.arange(n2), pos]
```

ACTUAL OUTPUT

```
row 0's similarities to rows 1, 2, 3 (cosine / 0.5): 0.0, 1.6, 1.2; its partner is row 2
by hand: -log(e^1.6 / (e^0.0 + e^1.6 + e^1.2)) = 0.6271
each row's probability of picking its partner: [0.5341, 0.0446, 0.5341, 0.0446]; the loss averages
    -log of these: 1.8680
Actual output from this lesson's run.
```

What it means

Row 0 is most similar to its partner (similarity 1.6 after dividing by τ), so it picks it with probability 0.5341: by hand –log of that is 0.6271. Rows 1 and 3 point in nearly opposite directions, so they pick each other with only 0.0446. Averaging all four gives 1.8680, the same number the exercise computes.

WHY CHOOSE THIS?

One formula: classification, with the batch's other views as the wrong answers.

WHY NOT THIS? (TRADEOFFS)

It needs many negatives in each batch; small batches give an easier, less informative task.

2. Correctness: the gradient, derived by hand

DEFINITION — start here, no prior knowledge assumed

The gradient of the loss has three steps: the softmax (probability minus target), the similarity matrix (which uses each vector twice), and the normalisation to unit length (which removes the part of the gradient along the vector itself).

REAL-WORLD SCENARIO

Checking a hand-drawn map by walking a few steps in each direction and seeing whether the height changes as the map says.

MATHEMATICS

$$\partial L/\partial S = (P - T) / 2N \qquad \partial L/\partial U = (G + G^{T})\, U / \tau,\ G = \partial L/\partial S$$

$$\partial L/\partial z = (\partial L/\partial u - u\,(u \cdot \partial L/\partial u)) / |z|$$

ACTUAL OUTPUT

```
the loss alone, 8 projections of 3 numbers: largest gap 1.5e-10
through a whole network (10-7-5-4, 3 images, 2 views each): largest gap 3.0e-10
Actual output from this lesson's run.
```

What it means

The formula agrees with nudging every number both ways: gaps of 1.5e-10 for the loss alone and 3.0e-10 through a whole small network, the same pipeline training uses. Both are rounding error.

WHY CHOOSE THIS?

A derived gradient, so training needs no autodiff library.

WHY NOT THIS? (TRADEOFFS)

Three chain-rule steps, each easy to get wrong; the check is essential.

3. The views

DEFINITION — start here, no prior knowledge assumed

Each view is the image turned by up to 15 degrees, shifted by up to 3 pixels, with an 8 x 8 patch blanked in half of the images, and light noise (0.1) added. Two independent random views are made of every image.

REAL-WORLD SCENARIO

Photos of the same house from slightly different spots, one with a van parked in front.

IMPLEMENTATION

```
def augment(X, rng, shift=3, angle=15, erase=8, noise=0.1):
    n = len(X); a = np.radians(rng.uniform(-angle, angle, n)); dx, dy = rng.randint(-shift, shift
        + 1, (2, n))
    r, c = np.mgrid[0:28, 0:28].reshape(2, -1) - 13.5
    cos, sin = np.cos(a)[:, None], np.sin(a)[:, None]
    src_r = np.rint(cos * r + sin * c + 13.5 - dy[:, None]).astype(int)
    src_c = np.rint(-sin * r + cos * c + 13.5 - dx[:, None]).astype(int)
    ok = (src_r >= 0) & (src_r < 28) & (src_c >= 0) & (src_c < 28)
    V = np.where(ok, X[np.arange(n)[:, None], np.clip(src_r, 0, 27) * 28 + np.clip(src_c, 0, 27)],
        0).astype(np.float32)
    if erase:
        r0, c0 = rng.randint(0, 28 - erase, (2, n)); hit = rng.uniform(size=n) < 0.5
        V = V.reshape(n, 28, 28)
        for i in np.where(hit)[0]:
            V[i, r0[i]:r0[i] + erase, c0[i]:c0[i] + erase] = 0
        V = V.reshape(n, -1)
    return np.clip(V + rng.normal(0, noise, V.shape).astype(np.float32), 0, 1)
```

ACTUAL OUTPUT

```
cosine similarity in raw pixels: two views of the same image 0.4232 on average; views of different
    images 0.3536
share of positive pairs less similar than the average negative pair: 0.3930
in pixel space many positive pairs look no more alike than strangers: the network has to learn
    what stays the same
Actual output from this lesson's run.
```

What it means

In raw pixels, two views of the same image have cosine similarity 0.4232 on average, barely above two different images' 0.3536; 0.3930 of positive pairs are less alike than the average negative pair. So the task cannot be solved by comparing pixels: the network has to learn what stays the same when an image is turned, moved and partly hidden.

WHY CHOOSE THIS?

The views define what the network learns to ignore.

WHY NOT THIS? (TRADEOFFS)

Views must be chosen by hand, with knowledge of the data (section 7).

4. Training

DEFINITION — start here, no prior knowledge assumed

A network 784 → 256 → 128 → 64. The 128-number layer is the representation; the last layer (the **projection head**) is used only for the loss and thrown away afterwards, as in SimCLR.

REAL-WORLD SCENARIO

Practising with a sparring partner whose moves you will never face again: the skill stays, the partner goes.

SPECIFICATION

Data	20,000 MNIST images, no labels
Batch	256 images = 512 views, so each view has 510 negatives
Temperature	0.1
Training	30 epochs, Adam at 0.001 (Day 126's)

IMPLEMENTATION

```
def train_contrastive(X, sizes, tau=0.5, epochs=20, batch=256, lr=1e-3, seed=0, aug=None):
    acts = ["relu"] * (len(sizes) - 2) + ["linear"]; net = d126.init_net(sizes, seed); opt =
        d126.Adam(net, lr)
    rng = np.random.RandomState(seed + 1); aug = aug or augment; hist = []
    for _ in range(epochs):
        o = rng.permutation(len(X)); losses = []
        for s in range(0, len(X) - batch + 1, batch):
            xb = X[o[s:s + batch]]; V = np.vstack([aug(xb, rng), aug(xb, rng)])
            A = d126.forward(net, V, acts); loss, dZ, _ = nt_xent(A[-1], tau)
            opt.step(net, d126.backward(net, A, acts, dZ)[0]); losses.append(loss)
        hist.append(float(np.mean(losses)))
    return net, hist
```

ACTUAL OUTPUT

```
loss every 5 epochs: [4.91, 2.694, 2.355, 2.154, 1.998, 1.9, 1.838]; trained in 120 s
for comparison: a network that could not tell views apart would score log(511) = 6.236
on 2,000 test images: cosine of positive pairs 0.7997, of negative pairs 0.0174
Actual output from this lesson's run.
```

What it means

The loss falls from 4.91 after the first epoch to 1.838, far below 6.236, the score of a network that cannot tell views apart. On test images it never saw, positive pairs end with cosine 0.7997 and negative pairs 0.0174: views of one image now point the same way, different images are nearly perpendicular. The loss was still falling at epoch 30.

WHY CHOOSE THIS?

Learns from unlabelled images alone, and from views no one had to label.

WHY NOT THIS? (TRADEOFFS)

Many negatives per step, so each epoch costs more than an autoencoder's.

5. What it learned: probes with few labels

DEFINITION — start here, no prior knowledge assumed

A **linear probe** trains a plain linear classifier (Day 116's softmax regression) on a frozen representation using a few labelled images. It measures how directly the representation exposes the digit. A kNN classifier and a probe with 100 labels per digit give two more views of the same question.

REAL-WORLD SCENARIO

Judging a filing system by how quickly a new clerk, given ten examples, can learn to find things in it.

IMPLEMENTATION

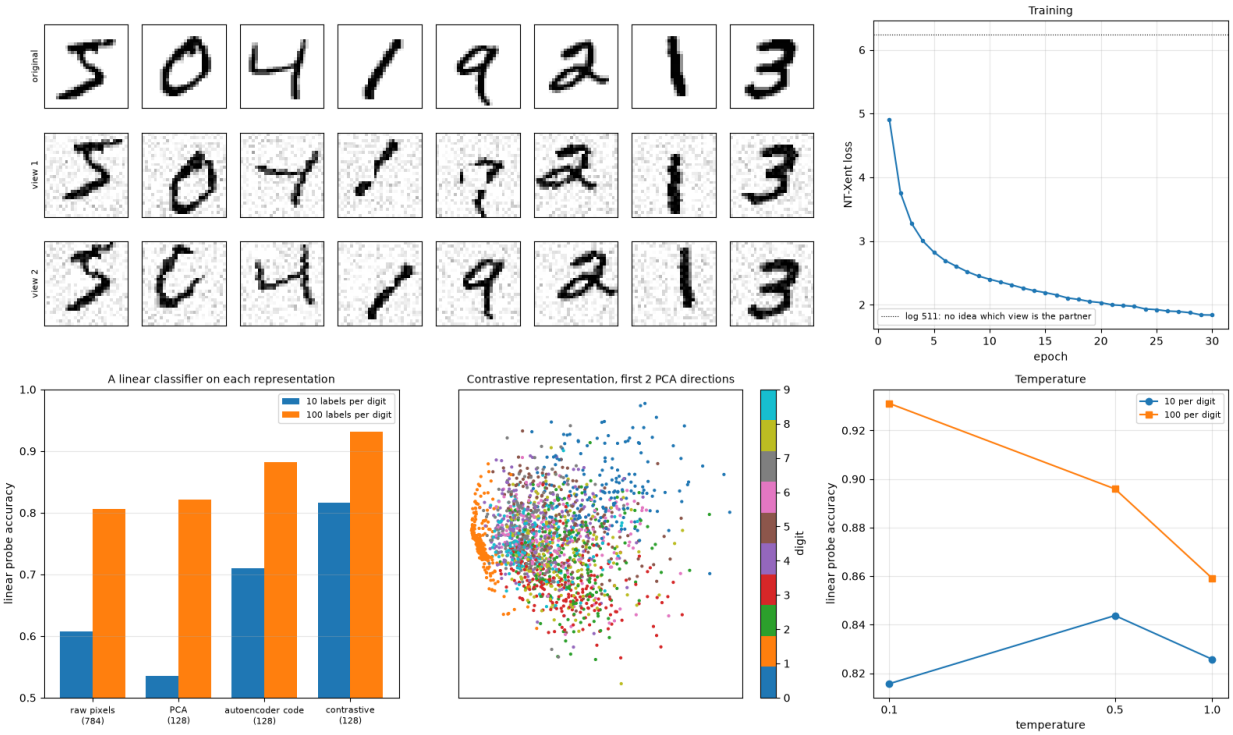
```
def probe(R, y, n_per, seed=0):
    rng = np.random.RandomState(seed)
    tr = np.concatenate([rng.choice(np.where(y[:60000] == c)[0], n_per, replace=False) for c in
        range(10)]); te = np.arange(60000, 70000)
    mu, sd = R[tr].mean(0), R[tr].std(0) + 1e-6
    W, b = d116.fit_softmax(((R[tr] - mu) / sd).astype(np.float64), y[tr], 10, lam=1e-2,
        steps=500)
    return float(np.mean((((R[te] - mu) / sd) @ W + b).argmax(1) == y[te]))
```

ACTUAL OUTPUT

```
  representation            kNN (5,000 labelled)  probe, 10 per digit  probe, 100 per digit
  raw pixels (784)                        0.9085               0.6078                0.8059
  PCA (128)                               0.9155               0.5356                0.8217
  autoencoder code (128)                  0.9110               0.7100                0.8824
  contrastive (128)                       0.9135               0.8157                0.9311
probes: Day 116's softmax regression on standardised features, tested on all 10,000 test images; 10 per digit
    averaged over 3 draws
Actual output from this lesson's run.
```

What it means

With 10 labels per digit the contrastive representation scores 0.8157, against 0.6078 for raw pixels, 0.5356 for PCA and 0.7100 for the autoencoder code; with 100 labels per digit, 0.9311 against 0.8059, 0.8217 and 0.8824. kNN with 5,000 labels hardly separates them (0.9085 to 0.9155): with many labels, nearest neighbours in pixel space already work. The advantage is largest exactly where labels are scarce. PCA with 10 labels per digit is below raw pixels here; it was not investigated further today.



Top: eight training images and two random views of each; the training loss. Bottom: probe accuracy for each representation; the contrastive representation of 2,000 test images on its first two PCA directions, coloured by digit; probe accuracy against temperature.

WHY CHOOSE THIS?

A representation that a few labels can use directly.

WHY NOT THIS? (TRADEOFFS)

kNN with many labels gains little; the representation is only as good as the views.

6. Temperature

DEFINITION — start here, no prior knowledge assumed

The temperature τ divides every similarity before the softmax. Small τ makes the loss focus on the hardest negatives (the most similar wrong views); large τ spreads attention over all of them.

REAL-WORLD SCENARIO

A strict judge who notices only the closest look-alikes, against a lenient one who weighs every candidate a little.

ACTUAL OUTPUT

```
   temperature  final loss  probe, 10 per digit  probe, 100 per digit
           0.1       1.838               0.8157                0.9311
           0.5       4.676               0.8437                0.8959
           1.0       5.410               0.8257                0.8591
(losses at different temperatures are on different scales and cannot be compared with each other)
Actual output from this lesson's run.
```

What it means

There is no single best temperature here. With 100 labels per digit, $\tau = 0.1$ is clearly best (0.9311 against 0.8959 and 0.8591). With 10 labels per digit, $\tau = 0.5$ is best (0.8437 against 0.8157 for 0.1). One training run per temperature, so small differences may be noise; the 100-label ordering is large. The final losses cannot be compared across temperatures, because τ changes the scale of the loss.

WHY CHOOSE THIS?

One number that strongly shapes what is learned.

WHY NOT THIS? (TRADEOFFS)

Its best value depends on what the representation is used for afterwards.

7. Which views matter

DEFINITION — start here, no prior knowledge assumed

The same training, with some of the changes switched off. Noise (0.1) stays on in every variant; the full set adds erasing.

REAL-WORLD SCENARIO

Teaching the dog lesson only with photos in different light: the child learns little about shape.

ACTUAL OUTPUT

```
  noise                     : probe, 10 per digit 0.5800; 100 per digit 0.8211
  noise + shift             : probe, 10 per digit 0.7724; 100 per digit 0.8482
  noise + shift + rotation  : probe, 10 per digit 0.8111; 100 per digit 0.8956
  (noise + shift + rotation + erasing: section 5's contrastive row)
Actual output from this lesson's run.
```

What it means

Noise alone teaches little: 0.5800 with 10 labels per digit, below raw pixels' 0.6078; a likely reason, not tested here, is that two noisy copies of one image can be matched without learning its shape. Adding shifts lifts it to 0.7724, rotation to 0.8111, and erasing (the full set) to 0.8157; with 100 labels per digit the steps are 0.8211, 0.8482, 0.8956 and 0.9311. Erasing adds little with 10 labels but 0.0355 with 100. The changes were added one at a time in this order, one run each, so the table shows what each step added on top of the previous ones, not what each change would do alone.

WHY CHOOSE THIS?

The views are where knowledge about the data enters.

WHY NOT THIS? (TRADEOFFS)

A poor choice of views gives a poor representation, and there is no label to warn you.

Interview preparation — technical questions

Q: Write the NT-Xent loss.

A: For each view, –log of the softmax probability of its partner among all other views, similarities being cosines over τ.

Q: Why throw the projection head away?

A: The loss shapes the head's output to ignore view details; the layer before it keeps more information. SimCLR found the layer before the head transfers better.

Q: How do you judge a self-supervised representation?

A: A linear probe with few labels: here 0.8157 with 10 per digit against 0.6078 for raw pixels.

Q: Why do batch sizes matter?

A: Every other view in the batch is a negative: 256 images gave 510 negatives per view.

Q: How do you check the hand-derived gradient?

A: Finite differences through the full network: largest gap 3.0e-10 here.

Interview preparation — theoretical questions

Q: What does the temperature do?

A: Sharpens or softens the softmax over negatives; here $\tau = 0.1$ was best with 100 labels per digit, 0.5 with 10.

Q: Why are the views so important?

A: The network learns to ignore exactly what the views change; section 7 measures what each change adds, added one at a time in a fixed order.

Q: How is contrastive learning different from an autoencoder?

A: It never rebuilds pixels; it only has to tell which views belong together.

Q: What can go wrong without negatives?

A: Every image could map to the same point and the positive pairs would still agree: a collapse the negatives prevent.

Key takeaway and what's next

> **KEY TAKEAWAY**
>
> Contrastive learning builds a representation by asking which random views belong together, with no labels and no reconstruction. Written from scratch here, it gave the best few-label classifier of the week so far, but its results hinge on two human choices: the temperature and, above all, the views.

- NT-Xent by hand: 1.8680; gradient checked to 3.0e-10.
- Probe with 10 labels per digit: contrastive 0.8157, autoencoder 0.7100, raw pixels 0.6078.
- Probe with 100 labels per digit: contrastive 0.9311, autoencoder 0.8824, raw pixels 0.8059.
- Temperature: 0.1 best with 100 labels per digit, 0.5 best with 10.

Suggested watch order, if starting from zero

- 1. Contrastive Learning with SimCLR | Deep Learning Animated (Deepia) — an animated walk through SimCLR's views, encoder, head and loss.
- 2. Contrastive Learning of visual representations - SimCLR (paper illustrated) (AI Bites) — the SimCLR paper's design choices, illustrated.

Day 132 preview

- **The Week 19 build**: a self-supervised pretext task (predicting rotations), compared with every representation of the week.